

Computational Macro

PhD-level course, University of Mannheim

Fabio Stohler*

Herbst-/Wintersemester 2026

10.09.2026 until 09.12.2026, Wednesdays from 23.09.2026

This is a PhD-level course on solution methods for (mainly) macroeconomic problems and models. Students first become familiar with seminal solution methods for dynamic problems before being introduced to state-of-the-art methods for tackling more complex problems. Building on a review of dynamic programming, the course covers different approaches to solving a household problem, namely value function iteration, policy function iteration, and the endogenous grid point method. It then embeds the household side in an aggregate environment by aggregating heterogeneous households. Finally, the course turns to models with aggregate dynamics, which are solved with local and global techniques, including neural network based methods.

Alongside the economics, the course teaches the craft of scientific computing as it is practiced in modern research teams. This covers version control with Git and GitHub, the productive and critical use of AI coding assistants, software engineering essentials such as code style, formatting, and project structure, and fully reproducible research workflows. Lectures are accompanied by in-class coding. Students regularly present and explain code to each other in a partially flipped classroom format.

*University of Mannheim, fabio.stohler@uni-mannheim.de.

1 Organization

From 23.09.2026 on, the course meets weekly on Wednesdays from 13:45 to 15:15 in room 002, L 9, until 09.12.2026. The first two sessions, on 10.09. and 17.09.2026, took place on Thursdays from 10:15 to 11:45 in room 211 (Seminarraum B 6, 30-32, Bauteil E-F).

Before the first meeting Students set up the course infrastructure before the first session. This means installing [VS Code](#), [Git](#), [GitHub Desktop](#), and [Julia](#) and/or Python through [Anaconda or Miniconda](#), and creating a [GitHub](#) account. Detailed installation instructions are provided on the course repository. Students are encouraged to register for a [GitHub Student Developer Pack](#), which gives free access to useful resources such as GitHub Copilot and private repositories.

Office hours Office hours are on Wednesdays from 10:00 to 12:00. Please write an email beforehand indicating your question such that I can prepare.

Course infrastructure All material is distributed through GitHub. Students maintain repositories for this course, to which all homework and problem set solutions are uploaded.

2 Learning objectives

After successfully completing the course, students are able to

- formulate dynamic economic problems recursively and derive analytical solutions in tractable benchmark cases,
- implement and compare the main solution methods for household problems and judge their accuracy and speed against analytical benchmarks,
- apply the core numerical tools behind these methods, in particular interpolation, function approximation, rootfinding, and numerical integration, as well as discretization and simulation of stochastic processes,
- compute stationary distributions of heterogeneous agent economies,
- solve models with aggregate risk using perturbation based and simulation based methods,
- train neural networks to solve high-dimensional dynamic models,
- collaborate on code through Git and GitHub and use AI coding assistants productively and critically,
- write readable, well-structured, automatically formatted code and package their work as a reproducible replication repository with pinned environments, random seeds, and run scripts.

3 Course format and interaction

A typical session consists of the introduction of new concepts, in-class coding, and discussion (not necessarily in this order). The four tools of the trade (version control, AI-assisted coding, software engineering, reproducibility) are not taught as separate standalone lectures. Each is introduced in a compact module embedded in the sessions of the first half of the semester and is then used continuously throughout the course.

The course uses several complementary forms of interaction. Coding homework comes as code templates with gaps that students fill in to get the code running, first for a model with a known analytical solution and then for a more general problem they solve themselves. Students upload their solutions to their individual course repository before class and are regularly asked to explain parts of their solution to each other in class. In addition, each student gives one short presentation on a “trivia” topic related to the course (see Section 6), which makes up 10% of the grade. Suggested topics for each week are highlighted in bold in the course plan. Short ungraded quizzes, in class and to take home, are used regularly to check understanding. Additional self-study material, such as short workbooks and primers for the following week, is distributed outside class.

4 Assessment

The final grade consists of problem sets (10%), a trivia presentation (10%), a midterm exam (40%), and a final project (40%). The format is designed to reduce the workload in the phases in which students are supposed to work on submissions for research proposals.

There are four problem sets, submitted in teams of two via GitHub, with deadlines in weeks 3, 5, 7, and 11 (see the course plan). Problem sets are due on the Sunday at 23:59 following the session of the respective week. The first problem sets are more concerned with pen-and-paper solutions and less with code, while the later problem sets reverse this priority. The pen-and-paper parts establish the analytical benchmarks against which we later compare our numerical solutions. The trivia presentation is a short presentation (about 10 minutes) on a topic chosen from the list in Section 6. The midterm takes place in class on 28.10.2026, lasts 90 minutes, and covers the first half of the course. Students who miss the midterm for a valid reason take an individually scheduled oral makeup exam covering the same material. In the final project, students solve and extend a model using the methods from the course. The deliverable is a reproducible GitHub repository plus a short write-up, due on Sunday, 13.12.2026, at 23:59. Projects are presented in the final session. Section 7 lists example projects. Detailed grading criteria are distributed in class.

AI coding assistants may be used for the problem sets and the final project. Their use must be briefly documented, and students must be able to explain every part of their submission. The midterm is written without AI tools.

5 Course plan

Tools modules are in italics, suggested trivia presentation topics in bold, and problem set deadlines are marked in the respective week.

#	Date	Topics
1	10.09.2026	Motivation and course overview. Review of dynamic programming. <i>Git and GitHub I: repositories, commits, push and pull, the course repo</i> . Infrastructure must be installed before this session.
2	17.09.2026	Value function iteration on-grid, for the deterministic growth model. <i>Git and GitHub II: branches, merge conflicts, pull requests, working in teams</i> .
3	23.09.2026	Numerical tools: interpolation (linear, cubic splines) and the maximization of a continuous function. Value function iteration off-grid. <i>AI-assisted coding: capabilities, failure modes, and course policy</i> . Contraction mappings: why is value function iteration a contraction? PS 1 due.
4	30.09.2026	The canonical consumption-savings problem. Policy function iteration. Markov chains, discretization and simulation of income processes. Comparing interpolation methods: linear, splines, Chebyshev .
5	07.10.2026	The endogenous grid point method. Occasionally binding constraints. <i>Software engineering: code style, formatting (black, JuliaFormatter), naming, functions, project structure, and performance</i> . Tauchen versus Rouwenhorst: which discretization when? PS 2 due.
6	14.10.2026	Aggregating the household side: policy functions as Markov chains, Young's method, stationary equilibrium (Huggett, Aiyagari). Beyond the standard endogenous grid method: generalizations and extensions .
7	21.10.2026	Writing efficient code and profiling. <i>Reproducibility: environments (Project.toml, requirements), seeds, replication packages</i> . Floating point arithmetic: why $0.1 + 0.2 \neq 0.3$. PS 3 due.
8	28.10.2026	Midterm exam (in class).
9	04.11.2026	Perturbation methods: linear rational expectations systems (Blanchard and Kahn, Klein). Random numbers and seeds: what does it take to make a simulation reproducible?
10	11.11.2026	Heterogeneous agents with aggregate risk I: Reiter's method and refinements. Solutions in the state space and in the sequence space. The Blanchard and Kahn conditions in practice .
11	18.11.2026	Heterogeneous agents with aggregate risk II: the Krusell and Smith algorithm. A tour of software packages for solving economic models, especially heterogeneous agent models . PS 4 due.
12	25.11.2026	Neural networks for model solutions I: deep learning foundations, all-in-one expectation operators (Maliar, Maliar and Winant). What is automatic differentiation?
13	02.12.2026	Neural networks for model solutions II: deep equilibrium nets, applications. Project feedback session.
14	09.12.2026	Project presentations and wrap-up.

6 Trivia presentation topics

Trivia presentation topics are chosen from the following list.

- Why is value function iteration a contraction? Blackwell's sufficient conditions, and which properties the value function inherits from the primitives (continuity, monotonicity, concavity, differentiability).
- Tauchen versus Rouwenhorst: which discretization method should you use when?
- Comparing interpolation methods: linear interpolation, splines, and Chebyshev polynomials.
- Beyond the standard endogenous grid method: generalizations and extensions.
- The Blanchard and Kahn conditions in practice: what goes wrong when they fail?
- A tour of existing repositories and toolboxes that help to solve computational models, especially heterogeneous agent models.
- What is automatic differentiation and how does it differ from numerical and symbolic differentiation?
- Floating point arithmetic: why $0.1 + 0.2 \neq 0.3$ and what this means for convergence criteria.
- Random numbers and seeds: what does it take to make a simulation reproducible?

7 Example projects

The following list gives examples of possible final projects.

- Extend the code for solving household consumption-savings models by a labor supply choice. Compare policies under KPR and GHH preferences.
- Compare the solutions of one model across different programming languages and software packages. Why can packages differ in their results?
- Set up and solve a household problem in which households choose between different assets.
- Approximate non-Gaussian income processes and compare different methods against each other.
- Solve a model to higher order.
- Replicate a paper of your own choosing.

8 Literature

Textbooks and general resources

- Heer, B. and Maußner, A. (2024). *Dynamic General Equilibrium Modeling: Computational Methods and Applications*. Springer.
- Fehr, H. and Kindermann, F. (2018). *Introduction to Computational Economics Using Fortran*. Oxford University Press.
- Ljungqvist, L. and Sargent, T. (2018). *Recursive Macroeconomic Theory*. 4th ed., MIT Press.
- Sargent, T. and Stachurski, J. QuantEcon lectures, <https://quantecon.org>.

Analytical benchmark

- Brock, W. A. and Mirman, L. J. (1972). Optimal economic growth and uncertainty: the discounted case. *Journal of Economic Theory* 4(3), 479–513.
- Angeletos, G.-M. (2007). Uninsured idiosyncratic investment risk and aggregate saving. *Review of Economic Dynamics* 10(1), 1–30.
- Acharya, S. and Dogra, K. (2020). Understanding HANK: insights from a PRANK. *Econometrica* 88(3), 1113–1158.
- Acharya, S., Challe, E., and Dogra, K. (2023). Optimal monetary policy according to HANK. *American Economic Review* 113(7), 1741–1782.

Markov chains and discretization

- Tauchen, G. (1986). Finite state Markov-chain approximations to univariate and vector autoregressions. *Economics Letters* 20(2), 177–181.
- Rouwenhorst, K. G. (1995). Asset pricing implications of equilibrium business cycle models. In: *Frontiers of Business Cycle Research*. Princeton University Press, 294–330.
- Kopecky, K. A. and Suen, R. M. (2010). Finite state Markov-chain approximations to highly persistent processes. *Review of Economic Dynamics* 13(3), 701–714.

Household problems and the endogenous grid method

- Carroll, C. D. (2006). The method of endogenous gridpoints for solving dynamic stochastic optimization problems. *Economics Letters* 91(3), 312–320.
- Barillas, F. and Fernández-Villaverde, J. (2007). A generalization of the endogenous grid method. *Journal of Economic Dynamics and Control* 31(8), 2698–2712.

- Hintermaier, T. and Koeniger, W. (2010). The method of endogenous gridpoints with occasionally binding constraints among endogenous variables. *Journal of Economic Dynamics and Control* 34(10), 2074–2088.

Aggregation and stationary equilibria

- Young, E. R. (2010). Solving the incomplete markets model with aggregate uncertainty using the Krusell-Smith algorithm and non-stochastic simulations. *Journal of Economic Dynamics and Control* 34(1), 36–41.

Perturbation methods and linear RE systems

- Blanchard, O. J. and Kahn, C. M. (1980). The solution of linear difference models under rational expectations. *Econometrica* 48(5), 1305–1311.
- Klein, P. (2000). Using the generalized Schur form to solve a multivariate linear rational expectations model. *Journal of Economic Dynamics and Control* 24(10), 1405–1423.

Heterogeneous agents with aggregate risk

- Krusell, P. and Smith, A. A. (1997). Income and wealth heterogeneity, portfolio choice, and equilibrium asset returns. *Macroeconomic Dynamics* 1(2), 387–422.
- Krusell, P. and Smith, A. A. (1998). Income and wealth heterogeneity in the macroeconomy. *Journal of Political Economy* 106(5), 867–896.
- Reiter, M. (2009). Solving heterogeneous-agent models by projection and perturbation. *Journal of Economic Dynamics and Control* 33(3), 649–665.
- Ahn, S., Kaplan, G., Moll, B., Winberry, T., and Wolf, C. (2017). When inequality matters for macro and macro matters for inequality. *NBER Macroeconomics Annual* 32, 1–75.
- Boppart, T., Krusell, P., and Mitman, K. (2018). Exploiting MIT shocks in heterogeneous-agent economies: the impulse response as a numerical derivative. *Journal of Economic Dynamics and Control* 89, 68–92.
- Bayer, C. and Luetticke, R. (2020). Solving heterogeneous agent models in discrete time with many idiosyncratic states by perturbation methods. *Quantitative Economics* 11(4), 1253–1288.
- Auclert, A., Bardóczy, B., Rognlie, M., and Straub, L. (2021). Using the sequence-space Jacobian to solve and estimate heterogeneous-agent models. *Econometrica* 89(5), 2375–2408.
- Bayer, C., Born, B., and Luetticke, R. (2024). Shocks, frictions, and inequality in US business cycles. *American Economic Review* 114(5), 1211–1247. (Distributional endogenous grid method.)

Neural networks for model solutions

- Maliar, L., Maliar, S., and Winant, P. (2021). Deep learning for solving dynamic economic models. *Journal of Monetary Economics* 122, 76–101.
- Azinovic, M., Gaegauf, L., and Scheidegger, S. (2022). Deep equilibrium nets. *International Economic Review* 63(4), 1471–1525.

Tools of the trade

- Gentzkow, M. and Shapiro, J. M. (2014). Code and data for the social sciences: a practitioner’s guide. University of Chicago, mimeo.
- Wilson, G. et al. (2017). Good enough practices in scientific computing. *PLOS Computational Biology* 13(6), e1005510.
- Chacon, S. and Straub, B. (2014). *Pro Git*. 2nd ed., Apress. Freely available at <https://git-scm.com/book>.
- von Gaudecker, H.-M. Effective programming practices for economists. University of Bonn course material, https://www.wiwi.uni-bonn.de/gaudecker/_static/epp_bonn/landing-page.html.